

1. a) Write a shell script program to find the Maximum three numbers .

```
#!/bin/bash
```

```
echo "Enter three numbers:"
```

```
read num1
```

```
read num2
```

```
read num3
```

```
max=$num1
```

```
if [ $num2 -gt $max ]; then
```

```
    max=$num2
```

```
fi
```

```
if [ $num3 -gt $max ]; then
```

```
    max=$num3
```

```
fi
```

```
echo "The maximum number is: $max"
```

Run: `chmod +x find_max.sh`

Output: Enter three numbers:

10

5

8

The maximum number is: 10

1. b) Write a shell script program for comparison of strings

```
#!/bin/bash
```

```
echo "Enter the first string:"
```

```
read str1
```

```
echo "Enter the second string:"
```

```
read str2
```

```
if [ "$str1" == "$str2" ]; then
```

```
    echo "Strings are equal."
```

```
elif [ "$str1" \< "$str2" ]; then
```

```
    echo "First string is less than the second string."
```

```
else
```

```
    echo "First string is greater than the second string."
```

```
fi
```

Run: `chmod +x compare_strings.sh`

Output: Enter the first string:

apple

Enter the second string:

banana

First string is less than the second string.

2.a) Perform Arithmetic operation using

CASE...

```
#!/bin/bash
```

```
echo "Enter the first number:"
```

```
read num1
```

```
echo "Enter the second number:"
```

```
read num2
```

```
echo "Enter the operation (+, -, *,
```

```
/):" read operator
```

```
case $operator in
```

```
  +)
```

```
    result=$((num1 + num2))
```

```
    echo "Result: $result"
```

```
    ;;
```

```
  -)
```

```
    result=$((num1 - num2))
```

```
    echo "Result: $result"
```

```
    ;;
```

```
  \*)
```

```
    result=$((num1 * num2))
```

```
    echo "Result: $result"
```

```
    ;;
```

```
  /)
```

```
    if [ $num2 -ne 0 ]; then
```

```
        result=$((num1 / num2))

        echo "Result: $result"

    else

        echo "Error: Division by zero is not allowed."

    fi

    ;;

*)

    echo "Invalid operator!"

    ;;

esac
```

Run: `chmod +x arithmetic_operations.sh`

Output: Enter the first number:

10

Enter the second number:

5

Enter the operation (+, -, *, /):

/

Result: 2.00

2. b) Write a program to draw a Pascal's

Triangle....

```
#!/bin/bash
```

```
echo "Enter the number of rows for Pascal's
```

```
Triangle:" read rows
```

```
factorial() {
```

```
    if [ $1 -le 1 ]; then
```

```
        echo 1
```

```
    else
```

```
        echo $(( $1 * $(factorial $(( $1 - 1 ))) )
```

```
    fi
```

```
}
```

```
binomial_coefficient() {
```

```
    local n=$1
```

```
    local r=$2
```

```
    local numerator=$(factorial $n)
```

```
    local denominator=$(( $(factorial $r) * $(factorial $(( $n - $r ))) )
```

```
    echo $(( numerator / denominator ))
```

```
}
```

```
for ((i = 0; i < rows; i++)); do
```

```
    for ((j = 0; j <= rows - i; j++)); do
```

```
        echo -n " "
```

```
    done
```

```
    for ((j = 0; j <= i; j++)); do
```

```
        coefficient=$(binomial_coefficient $i $j)

        echo -n "$coefficient "

    done

    echo
done

Run: chmod +x pascals_triangle.sh
```

Output: Enter the number of rows for Pascal's Triangle: 5

```
1
1 1
1 2 1
1 3 3 1
1 4 6 4 1
```

3 a. Calculate the factorial value of a number using shell script .

```
#!/bin/bash

echo "Enter a number:"

read num

factorial=1

if [ $num -lt 0 ]; then

echo "Factorial is not defined for negative numbers."

elif [ $num -eq 0 ]; then

echo "Factorial of 0 is 1."

else

for ((i = 1; i <= num; i++)); do

factorial=$((factorial * i))

done

echo "Factorial of $num is $factorial."

fi
```

Run: chmod +x factorial.sh

Output: Enter a number: 5

Factorial of 5 is 120.

3 b. To write a shell program to generate fibonacci series.

```
#!/bin/bash

echo "Enter the number of Fibonacci terms to generate:"

read n

a=0

b=1

if [ $n -eq 0 ]; then

echo "No terms to generate."

elif [ $n -eq 1 ]; then

echo "Fibonacci series:"

echo "$a"

else

echo "Fibonacci series:"

echo "$a"

echo "$b"

for ((i = 2; i < n; i++)); do

next=$((a + b))

echo "$next"

a=$b

b=$next

done

fi
```

Output : Enter the number of Fibonacci terms to generate: 8

Fibonacci series:

0

1

1

2

3

5

8

13

4 a. Write a program to demonstrates a one-way pipe between two Process .

```
#!/bin/bash
```

```
# Create a named pipe (FIFO)
```

```
pipe_file="my_pipe"
```

```
mkfifo $pipe_file
```

```
producer_process() {
```

```
  for i in {1..10}; do
```

```
    echo "Producer: Sending data $i to the pipe"
```

```
    echo "$i" > $pipe_file
```

```
    sleep 1
```

```
  done
```

```
  echo "Producer: Finished sending data"
```

```
  exit 0
```

```
}
```

```
consumer_process() {
```

```
  while true; do
```

```
    if read -r line < $pipe_file; then
```

```
      echo "Consumer: Received data: $line"
```

```
    fi
```

```
  done
```

```
}
```

```
Run : chmod +x pipe_demo.sh
```

4 b. Write a program to illustrate IPC through pipe and fork system calls – Printing only odd numbers.

```
#include <stdio.h>
#include <unistd.h>
#include <stdlib.h>

int main() {
    int pipe_fd[2];
    pid_t pid;
    // Create a pipe
    if (pipe(pipe_fd) == -1) {
        perror("Pipe creation failed");
        exit(EXIT_FAILURE);
    }
    pid = fork();

    if (pid < 0) {
        perror("Fork failed");
        exit(EXIT_FAILURE);
    }

    if (pid == 0) {
        close(pipe_fd[1]);

        int number;
        printf("Child process (printing odd numbers):\n");

        while (read(pipe_fd[0], &number, sizeof(number)) > 0) {
            if (number % 2 != 0) {
                printf("%d\n", number);
            }
        }

        close(pipe_fd[0]);
        exit(EXIT_SUCCESS);
    } else {
        close(pipe_fd[0]);

        printf("Parent process (sending numbers to child):\n");

        for (int i = 1; i <= 10; i++) {
            printf("Sending: %d\n", i);
            write(pipe_fd[1], &i, sizeof(i));
        }
    }
}
```

```
    }  
    close(pipe_fd[1]);  
    wait(NULL);  
  
    return 0;  
}
```

Run:

Output :

Parent process (sending numbers to child):

Sending: 1

Sending: 2

Sending: 3

Sending: 4

Sending: 5

Sending: 6

Sending: 7

Sending: 8

Sending: 9

Sending: 10

Child process (printing odd numbers):

1

3

5

7

9

From here onwards its C program

5 a. To write a program to create a process in LINUX.

```
#include <stdio.h>
#include <unistd.h>
int main() {
    pid_t child_pid;
    child_pid = fork();
    if (child_pid < 0) {
        perror("Fork failed");
        return 1;
    }
    if (child_pid == 0) {
        printf("This is the child process (PID %d).\n", getpid()); } else {
        printf("This is the parent process (PID %d).\n", getpid()); printf("Child process created with PID %d.\n", child_pid); }
    return 0;
}
```

Run: gcc create_process.c

Output: This is the parent process (PID xxxx).

Child process created with PID yyyy.

This is the child process (PID yyyy).

5 b. To study Dinning Philosophers Problem.

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>
#define NUM_PHILOSOPHERS 5
pthread_mutex_t forks[NUM_PHILOSOPHERS];
void *philosopher(void *arg) {
    int id = *(int *)arg;
    int left_fork = id;
    int right_fork = (id + 1) % NUM_PHILOSOPHERS;
    while (1) {
        (sleep)
        printf("Philosopher %d is thinking.\n", id);
        sleep(2);
        pthread_mutex_lock(&forks[left_fork]);
        printf("Philosopher %d picked up the left fork (%d).\n", id, left_fork);
        pthread_mutex_lock(&forks[right_fork]);
        printf("Philosopher %d picked up the right fork (%d).\n", id, right_fork);
        (sleep)
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
        pthread_mutex_unlock(&forks[left_fork]);
        printf("Philosopher %d put down the left fork (%d).\n", id, left_fork);
        pthread_mutex_unlock(&forks[right_fork]);
        printf("Philosopher %d put down the right fork (%d).\n", id, right_fork);
    }
}

int main() {
    pthread_t philosophers[NUM_PHILOSOPHERS];
    int philosopher_ids[NUM_PHILOSOPHERS];
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_mutex_init(&forks[i], NULL);
    }
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        philosopher_ids[i] = i;
        pthread_create(&philosophers[i], NULL, philosopher, &philosopher_ids[i]);
    }
    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        pthread_join(philosophers[i], NULL);
    }
    return 0;
}
```

///// Run: gcc dining_philosophers.c/////

6. Write a program to implement the following process scheduling algorithms

i. First Come First Serve

ii. Shortest Remaining Job First

iii. Round Robin

i. FCFS

```
#include <stdio.h>
struct Process {
int id;
int arrival;
int burst;
int waiting;
int turnaround;
};
void calculateWaitingTime(struct Process processes[], int n) { int totalWaiting = 0;
processes[0].waiting = 0;
for (int i = 1; i < n; i++) {
processes[i].waiting = processes[i - 1].waiting + processes[i - 1].burst; totalWaiting +=
processes[i].waiting;
}
float avgWaiting = (float)totalWaiting / n;
printf("Average Waiting Time: %.2f\n", avgWaiting);
}
void calculateTurnaroundTime(struct Process processes[], int n) {
int totalTurnaround = 0;
for (int i = 0; i < n; i++) {
processes[i].turnaround = processes[i].waiting + processes[i].burst;
totalTurnaround += processes[i].turnaround;
}

float avgTurnaround = (float)totalTurnaround / n;
printf("Average Turnaround Time: %.2f\n", avgTurnaround);
}
int main() {
int n;
printf("Enter the number of processes: ");
scanf("%d", &n);
```

```

struct Process processes[n];
for (int i = 0; i < n; i++) {
    processes[i].id = i + 1;
    printf("Enter arrival time for Process %d: ", i + 1);
    scanf("%d", &processes[i].arrival);
    printf("Enter burst time for Process %d: ", i + 1);
    scanf("%d", &processes[i].burst);
}
for (int i = 0; i < n - 1; i++) {
    for (int j = 0; j < n - i - 1; j++) {
        if (processes[j].arrival > processes[j + 1].arrival)
        {
            struct Process temp = processes[j];
            processes[j] = processes[j + 1];
            processes[j + 1] = temp;
        }
    }
}
calculateWaitingTime(processes, n);
calculateTurnaroundTime(processes, n);
printf("Process\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\n",
        processes[i].id, processes[i].arrival, processes[i].burst, processes[i].waiting,
        processes[i].turnaround);
}
return 0;
}

```

II. Shortest job first

```
#include <stdio.h>

struct Process {
    int id;
    int arrival;
    int burst;
    int waiting;
    int turnaround;
};

void calculateWaitingTime(struct Process processes[], int n) {
    int currentTime = 0;
    int totalWaiting = 0;
    for (int i = 0; i < n; i++) {
        int shortest = -1;
        int minBurst = 9999;
        for (int j = 0; j < n; j++) {
            if (processes[j].arrival <= currentTime && processes[j].burst < minBurst &&
                processes[j].burst > 0) {
                shortest = j;
                minBurst = processes[j].burst;
            }
        }
        if (shortest == -1) {
            currentTime++;
            i--;
        }
    }
}
```

```

continue;
}

// Execute the selected process

processes[shortest].waiting = currentTime - processes[shortest].arrival;
processes[shortest].turnaround = processes[shortest].waiting + processes[shortest].burst;

totalWaiting += processes[shortest].waiting;

currentTime += processes[shortest].burst;

processes[shortest].burst = 0;

}

float avgWaiting = (float)totalWaiting / n;

printf("Average Waiting Time: %.2f\n", avgWaiting);

}

void calculateTurnaroundTime(struct Process processes[], int n) {

for (int i = 0; i < n; i++) { processes[i].turnaround = processes[i].waiting + processes[i].burst;
}

}

int main() {

int n;

printf("Enter the number of processes: ");

scanf("%d", &n);

struct Process processes[n];

for (int i = 0; i < n; i++) {

processes[i].id = i + 1;

printf("Enter arrival time for Process %d: ", i + 1);

scanf("%d", &processes[i].arrival);

printf("Enter burst time for Process %d: ", i + 1);

scanf("%d", &processes[i].burst);

}

calculateWaitingTime(processes, n);

calculateTurnaroundTime(processes, n);

```

```
printf("Process\tArrival Time\tBurst Time\tWaiting Time\tTurnaround Time\n");  
for (int i = 0; i < n; i++) {  
    printf("%d\t%d\t\t%d\t\t%d\t\t%d\n",  
    processes[i].id, processes[i].arrival, processes[i].burst, processes[i].waiting,  
    processes[i].turnaround);  
}  
return 0;  
}
```

iii. Round Robin

```
#include <stdio.h>

struct Process {
    int process_id;
    int arrival_time;
    int burst_time;
    int remaining_time;
};

void findWaitingTime(struct Process processes[], int n, int quantum, int waiting_time[]) {
    int remaining_burst[n];
    for (int i = 0; i < n; i++) {
        remaining_burst[i] = processes[i].burst_time;
    }

    int t = 0;
    while (1) {
        int done = 1;
        for (int i = 0; i < n; i++) {
            if (remaining_burst[i] > 0) {
                done = 0;
                if (remaining_burst[i] > quantum) {
                    t += quantum;
                    remaining_burst[i] -= quantum;
                } else {
                    t = t + remaining_burst[i];
                    waiting_time[i] = t - processes[i].burst_time;
                    remaining_burst[i] = 0;
                }
            }
        }
    }
}
```

```

}

}

}

if (done == 1) {

break;

}

}

}

void findTurnAroundTime(struct Process processes[], int n, int waiting_time[], int
turnaround_time[]) {

for (int i = 0; i < n; i++) {

turnaround_time[i] = processes[i].burst_time + waiting_time[i]; }

}

void findAverageTime(struct Process processes[], int n, int quantum) {

int waiting_time[n], turnaround_time[n];

findWaitingTime(processes, n, quantum, waiting_time);

findTurnAroundTime(processes, n, waiting_time, turnaround_time);

printf("Process ID\tBurst Time\tWaiting Time\tTurnaround Time\n");

int total_waiting_time = 0, total_turnaround_time = 0;

for (int i = 0; i < n; i++) {

total_waiting_time += waiting_time[i];

total_turnaround_time += turnaround_time[i];

printf("%d\t%d\t%d\t%d\n", processes[i].process_id,

processes[i].burst_time, waiting_time[i], turnaround_time[i]); }

printf("\nAverage waiting time = %f\n", (float)total_waiting_time / (float)n);

printf("Average turnaround time = %f\n", (float)total_turnaround_time / (float)n);

}

int main() {

```

```
int n, quantum;

printf("Enter the number of processes: ");

scanf("%d", &n);

struct Process processes[n];

printf("Enter the quantum time for Round Robin: ");

scanf("%d", &quantum);

printf("Enter arrival time and burst time for each process:\n");

for (int i = 0; i < n; i++) {

printf("Process %d:\n", i + 1);

processes[i].process_id = i + 1;

printf("Arrival time: ");

scanf("%d", &processes[i].arrival_time);

printf("Burst time: ");

scanf("%d", &processes[i].burst_time);

processes[i].remaining_time = processes[i].burst_time;

}

findAverageTime(processes, n, quantum);

return 0;

}
```

7. Write a program To simulate banker's algorithm for deadlock avoidance.

```
#include <stdio.h>

#define MAX_PROCESSES 10
#define MAX_RESOURCES 10

int available[MAX_RESOURCES];
int max[MAX_PROCESSES][MAX_RESOURCES];
int allocation[MAX_PROCESSES][MAX_RESOURCES];
int need[MAX_PROCESSES][MAX_RESOURCES];
int n, m;

int requestResource(int process, int request[]) {
    for (int i = 0; i < m; i++) {
        if (request[i] > need[process][i]) {
            printf("Error: Request exceeds maximum need for process %d\n", process);
            return 0;
        }
    }

    for (int i = 0; i < m; i++) {
        if (request[i] > available[i]) {
            printf("Process %d must wait since resources are not available\n", process);
            return 0;
        }
    }

    for (int i = 0; i < m; i++) {
        available[i] -= request[i];
        allocation[process][i] += request[i];
        need[process][i] -= request[i];
    }

    if (isSafe()) {
```

```

printf("Resource request granted to process %d\n", process);

return 1;

} else {

printf("Resource request denied to maintain the system in a safe state\n");

for (int i = 0; i < m; i++) {

available[i] += request[i];

allocation[process][i] -= request[i]; need[process][i] += request[i];

}

return 0;

}

}

int isSafe() {

int work[MAX_RESOURCES]; int finish[MAX_PROCESSES];

for (int i = 0; i < m; i++) {

work[i] = available[i];

}

for (int i = 0; i < n; i++) {

finish[i] = 0;

}

int safeSequence[n];

int count = 0;

while (count < n) {

int found = 0;

for (int i = 0; i < n; i++) {

if (finish[i] == 0) {

int j;

for (j = 0; j < m; j++) {

if (need[i][j] > work[j]) {

break;

}

}

}

}

```

```

}

if (j == m) {

for (j = 0; j < m; j++) {

work[j] += allocation[i][j];

}

safeSequence[count] = i;

finish[i] = 1;

count++;

found = 1;

}

}

}

if (found == 0) {

return 0;

}

}

return 1;

}

int main() {

printf("Enter the number of processes: ");

scanf("%d", &n);

printf("Enter the number of resources: ");

scanf("%d", &m);

printf("Enter the available resources: ");

for (int i = 0; i < m; i++) {

scanf("%d", &available[i]);

}

printf("Enter the maximum demand for each process:\n");

for (int i = 0; i < n; i++) {

for (int j = 0; j < m; j++) {

```

```

scanf("%d", &max[i][j]);
need[i][j] = max[i][j];
}
}

printf("Enter the allocation of resources for each process:\n");
for (int i = 0; i < n; i++) {
    for (int j = 0; j < m; j++) {
        scanf("%d", &allocation[i][j]);
        need[i][j] -= allocation[i][j];
        available[j] -= allocation[i][j];
    }
}

int request[MAX_RESOURCES];
int process;

printf("Enter the process number (0 to %d) requesting resources: ", n - 1);
scanf("%d", &process);

printf("Enter the resource request for process %d: ", process);
for (int i = 0; i < m; i++) {
    scanf("%d", &request[i]);
}

if (process < 0 || process >= n) {
    printf("Invalid process number\n");
} else {
    if (requestResource(process, request)) {
        printf("Resource allocation successful\n");
    } else {
        printf("Resource allocation failed\n");
    }
}

return 0; }

```

8. Write a program To Page replacement algorithm for FIFO.

```
#include <stdio.h>

#include <stdbool.h>

#define MAX_FRAMES 10

bool isPagePresent(int page, int frames[], int frameSize) {
    for (int i = 0; i < frameSize; i++) {
        if (frames[i] == page) {
            return true;
        }
    }
    return false;
}

int fifoPageReplacement(int pages[], int pageReferenceLength, int capacity) {
    int frames[MAX_FRAMES];
    int pageFaults = 0;
    int frameIndex = 0;
    for (int i = 0; i < MAX_FRAMES; i++) {
        frames[i] = -1;
    }
    for (int i = 0; i < pageReferenceLength; i++) {
        if (!isPagePresent(pages[i], frames, capacity)) {
            frames[frameIndex] = pages[i];
            frameIndex = (frameIndex + 1) % capacity;
            pageFaults++;
        }
        printf("Page %d: ", pages[i]);
        for (int j = 0; j < capacity; j++) {
            if (frames[j] == -1) {
                printf(" | ");
            } else {
                printf(" %d", frames[j]);
            }
        }
        printf("\n");
    }
}
```

```
printf("%d | ", frames[j]);  
}  
}  
printf("\n");  
}  
return pageFaults;  
}  
  
int main() {  
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};  
    int pageReferenceLength = sizeof(pages) / sizeof(pages[0]);  
    int capacity = 4;  
    int pageFaults = fifoPageReplacement(pages, pageReferenceLength, capacity);  
    printf("\nTotal page faults using FIFO: %d\n", pageFaults);  
    return 0;  
}
```

9. Write a program To Page replacement algorithm for LRU.

```
#include <stdio.h>

#include <stdbool.h>

#define MAX_FRAMES 10

int findLRU(int usedTime[], int frameSize) {

    int min = usedTime[0];

    int index = 0;

    for (int i = 1; i < frameSize; i++) {

        if (usedTime[i] < min) {

            min = usedTime[i];

            index = i;

        }

    }

    return index;

}

isPagePresent(int page, int frames[], int frameSize) {

    for (int i = 0; i < frameSize; i++) {

        if (frames[i] == page) {

            return true;

        }

    }

    return false;

}

int lruPageReplacement(int pages[], int pageReferenceLength, int capacity) { int
frames[MAX_FRAMES];

    int usedTime[MAX_FRAMES] = {0}; int pageFaults = 0;

    for (int i = 0; i < MAX_FRAMES; i++) {

        frames[i] = -1; }

    for (int i = 0; i < pageReferenceLength; i++) {

        if (!isPagePresent(pages[i], frames, capacity)) {
```

```

int emptyIndex = -1;
for (int j = 0; j < capacity; j++) {
    if (frames[j] == -1) {
        emptyIndex = j;
        break;
    }
}

if (emptyIndex != -1) {
    frames[emptyIndex] = pages[i];
    usedTime[emptyIndex] = i; } else {
    int leastUsedIndex = findLRU(usedTime, capacity); frames[leastUsedIndex] = pages[i];
    usedTime[leastUsedIndex] = i; }
pageFaults++;
} else {
    for (int j = 0; j < capacity; j++) {
        if (frames[j] == pages[i]) {
            usedTime[j] = i;
            break;
        }
    }
}

printf("Page %d: ", pages[i]);
for (int j = 0; j < capacity; j++) {
    if (frames[j] == -1) {
        printf(" | ");
    } else {
        printf("%d | ", frames[j]);
    }
}

printf("\n");

```

```
}  
  
return pageFaults;  
  
}  
  
int main() {  
  
int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};  
  
int pageReferenceLength = sizeof(pages) / sizeof(pages[0]);  
  
int capacity = 4;  
  
int pageFaults = lruPageReplacement(pages, pageReferenceLength, capacity); printf("\nTotal page  
faults using LRU: %d\n", pageFaults);  
  
return 0;  
  
}
```

10. Write a program To Page replacement algorithm for LFU.

```
#include <stdio.h>

#include <stdbool.h>

#define MAX_FRAMES 10

int findLFU(int frequency[], int frameSize) {
    int min = frequency[0];
    int index = 0;
    for (int i = 1; i < frameSize; i++) {
        if (frequency[i] < min) {
            min = frequency[i];
            index = i;
        }
    }
    return index;
}

isPagePresent(int page, int frames[], int frameSize) { for (int i = 0; i < frameSize; i++) {
    if (frames[i] == page) {
        return true;
    }
}
return false;
}

int lfuPageReplacement(int pages[], int pageReferenceLength, int capacity) { int
frames[MAX_FRAMES];

int frequency[MAX_FRAMES] = {0};

int pageFaults = 0;

for (int i = 0; i < MAX_FRAMES; i++) {
    frames[i] = -1;
}

for (int i = 0; i < pageReferenceLength; i++) {
```

```

if (!isPagePresent(pages[i], frames, capacity)) {
    int emptyIndex = -1;
    for (int j = 0; j < capacity; j++) {
        if (frames[j] == -1) {
            emptyIndex = j;
            break;
        }
    }
    if (emptyIndex != -1) {
        frames[emptyIndex] = pages[i];
        frequency[emptyIndex] = 1;
    } else {
        int leastFrequentIndex = findLFU(frequency, capacity); frames[leastFrequentIndex] = pages[i];
        frequency[leastFrequentIndex] = 1;
    }
    pageFaults++;
} else {
    for (int j = 0; j < capacity; j++) {
        if (frames[j] == pages[i]) {
            frequency[j]++;
            break;
        }
    }
}
printf("Page %d: ", pages[i]);
for (int j = 0; j < capacity; j++) {
    if (frames[j] == -1) {
        printf(" | ");
    } else {
        printf("%d | ", frames[j]);
    }
}

```

```
}  
  
}  
  
printf("\n");  
  
}  
  
return pageFaults;  
  
}  
  
int main() {  
  
    int pages[] = {7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2};  
  
    int pageReferenceLength = sizeof(pages) / sizeof(pages[0]);  
  
    int capacity = 4;  
  
    int pageFaults = lfuPageReplacement(pages, pageReferenceLength, capacity); printf("\nTotal page  
faults using LFU: %d\n", pageFaults);  
  
    return 0;  
  
}
```